

Лекция 4. Организация защищенных каналов: OpenVPN Server

Цель лекции

Понять назначение Remote Access VPN на примере OpenVPN

Разобрать базовые роли OpenVPN Server, OpenVPN Client, tunnel и TUN-интерфейса

Настроить простой OpenVPN-доступ к внутренней сети через сертификаты

Обеспечить маршрутизацию, NAT и firewall только для разрешенного сервиса

Проверить, что HTTPS к web01 доступен, а SSH к web01 остается запрещен

Учебные вопросы

1. VPN, Remote Access и Site-to-Site
2. OpenVPN Server/Client, Tunnel и TUN
3. Базовая PKI: CA, server certificate, client certificate
4. Server config, Client config, routes и DNS
5. IP forwarding, NAT и firewall
6. Certificate revocation и диагностика

Основной сценарий всей лекции

vpn-client: 172.16.100.50

vpn01 external: 172.16.100.10, UDP 1194

vpn01 tun0: 10.8.0.1, VPN subnet 10.8.0.0/24

vpn01 internal: 192.168.30.1, Internal LAN
192.168.30.0/24

web01: 192.168.30.10, HTTPS 443

Задача: client01 через OpenVPN получает HTTPS-доступ
к web01, но SSH к web01 запрещен

Топология: vpn-client — vpn01 — Internal LAN — web01

Remote Access VPN: защищенный доступ клиента к внутреннему серверу web01



Путь к web01 (пример)

1. Клиент подключается к `vpn01:1194/UDP`
2. Устанавливается TLS-сессия, аутентификация по сертификату
3. Создается `tun0 (10.8.0.0/24)`
4. Клиент отправляет трафик к `192.168.30.10` через туннель
5. `vpn01` маршрутизирует пакет во внутреннюю сеть
6. `web01` обрабатывает запрос и отправляет ответ обратно
7. Ответ возвращается через `vpn01` по туннелю клиенту

Адресный план

- `vpn-client` (внешний): `172.16.100.50/D4`
- `vpn01 enp0s3` (внешний): `172.16.100.10/24`
- `vpn01 enp0s8` (внутренний): `192.168.30.1/24`
- `web01` (внутренний): `192.168.30.10/24`
- VPN tunnel (`tun0`): `10.8.0.0/24`

Условные обозначения

- VPN-туннель (шифрование, аутентификация, целостность)
- Физическое соединение
- 🔒 Защищенное VPN-соединение
- 🔄 Коммутатор / внутренняя сеть



Важно: VPN не отменяет firewall, учет доступа и принцип минимально необходимого доступа.

1. VPN создает защищенный канал поверх чужой сети

VPN (Virtual Private Network) - технология защищенного соединения поверх недоверенной сети
VPN не делает внутренний сервис автоматически безопасным

VPN дает путь к внутренней сети, а правила доступа задаются маршрутизацией и firewall

OpenVPN использует TLS и сертификаты для защиты и проверки сторон

В этой лекции цель не весь корпоративный VPN, а один понятный Remote Access-стенд

Remote Access и Site-to-Site решают разные задачи

Remote Access - один пользователь, ноутбук или клиентская VM подключается к сети организации

Site-to-Site - VPN соединяет две сети, например офис и филиал

В Remote Access клиент получает маршрут к внутренним ресурсам

В Site-to-Site маршрутизаторы или серверы обычно обмениваются маршрутами между подсетями

Основная лаборатория этой лекции - только Remote Access

OpenVPN, WireGuard и IPsec выбирают под сценарий

OpenVPN - TLS, сертификаты, удобен для Remote Access и учебного разбора

WireGuard - public/private keys и простая конфигурация

IPsec - часто используется на маршрутизаторах и firewall

Сравнение не является главной темой лекции

В лаборатории используется OpenVPN, потому что он хорошо показывает роли сервера, клиента и сертификатов

UDP 1194 - типовой транспорт OpenVPN

port 1194 - стандартный порт OpenVPN

proto udp - использовать UDP как транспортный протокол

Для OpenVPN UDP обычно предпочтителен из-за меньших накладных расходов

TCP over TCP может создавать дополнительные задержки при потерях пакетов

Для базовой лаборатории используем UDP 1194 и не усложняем транспорт

2. OpenVPN строит tunnel между client и server

OpenVPN Server - vpn01, принимает подключения клиентов

OpenVPN Client - vpn-client, подключается к серверу

Tunnel - логический защищенный канал поверх обычной IP-сети

Через tunnel передаются пакеты к внутренней сети 192.168.30.0/24

После подключения у клиента появляется виртуальный VPN-интерфейс

TUN и TAP работают на разных уровнях

TUN - Layer 3, передает IP packets

TUN использует routing и подходит для большинства Remote Access-сценариев

TAP - Layer 2, передает Ethernet frames

TAP может применяться для bridging, но сложнее для базовой лаборатории

В этой лекции используем `dev tun`

TUN делает VPN похожим на отдельную маршрутизируемую сеть

VPN-клиенты получают адреса из 10.8.0.0/24

vpn01 получает адрес tun0 10.8.0.1

client01 отправляет пакеты к 192.168.30.0/24 через
VPN-интерфейс

vpn01 пересылает пакеты между tun0 и внутренним
интерфейсом

Поэтому далее нужны route, ip_forward, NAT и firewall

TUN-туннель и маршрут к внутренней сети



Без маршрута и IP forwarding доступ к LAN не появится



Главная идея

- ✓ TUN = L3-туннель
- ✓ Клиент получает VPN-адрес
- ✓ Доступ к LAN идет через маршрут

3. Базовая PKI подтверждает сервер и клиента

PKI (Public Key Infrastructure) - инфраструктура открытых ключей и сертификатов

CA (Certificate Authority) - центр сертификации, который подписывает сертификаты

Server Certificate - сертификат, подтверждающий vpn01

Client Certificate - сертификат, подтверждающий client01

Private Key - закрытый ключ владельца сертификата, его нельзя передавать другим

Главная идея: доверяем не паролю, а сертификату, подписанному CA

Каждый клиент должен иметь свой certificate и private key

client01 получает свой client certificate

client01 получает свой private key

Один общий сертификат для всех клиентов
использовать нельзя

Уникальный сертификат позволяет отозвать доступ
одного клиента

Уникальный сертификат упрощает аудит: видно, какой
клиент подключался

Сертификаты должны использоваться по назначению

Серверный сертификат должен использоваться как серверный

Клиентский сертификат должен использоваться как клиентский

remote-cert-tls server - клиент проверяет, что подключается к серверу

remote-cert-tls client - сервер проверяет, что подключается клиент

Глубокий разбор serverAuth и clientAuth относится к отдельной теме PKI

Дополнительно: CA private key лучше хранить отдельно

CA private key подписывает новые сертификаты

Если CA private key украден, злоумышленник сможет выпускать доверенные сертификаты

В production CA private key желательно хранить отдельно от VPN-сервера

В учебной лаборатории допускается упрощенная схема, но риск нужно понимать

Закрытые ключи клиентов и сервера защищают правами файловой системы

tls-crypt защищает управляющий канал OpenVPN

tls-crypt /etc/openvpn/server/ta.key - подключает дополнительную защиту управляющего канала
ta.key должен быть одинаково доступен серверу и клиентскому профилю

tls-crypt помогает скрыть и защитить часть TLS-обмена OpenVPN

В этой лекции используем tls-crypt без подробного сравнения с tls-auth

Файл ta.key также относится к чувствительным данным

4. `server.conf` начинается с транспорта и TUN

```
port 1194  
proto udp  
dev tun  
server 10.8.0.0 255.255.255.0  
topology subnet
```

Эти строки задают порт, транспорт, тип туннеля и VPN-подсеть

server.conf подключает PKI и политику клиента

```
ca /etc/openvpn/server/ca.crt  
cert /etc/openvpn/server/vpn01.crt  
key /etc/openvpn/server/vpn01.key  
tls-crypt /etc/openvpn/server/ta.key  
remote-cert-tls client
```

Эти строки задают доверенный СА, сертификат сервера, закрытый ключ и проверку клиентского сертификата

server.conf передает клиенту маршрут к LAN

```
push "route 192.168.30.0 255.255.255.0"
```

```
keepalive 10 120
```

```
persist-key
```

```
persist-tun
```

```
verb 3
```

push route сообщает клиенту, что сеть 192.168.30.0/24
доступна через VPN

Криптографические директивы зависят от версии OpenVPN

ecdh-curve, data-ciphers и data-ciphers-fallback не являются главной темой этой лекции

Набор cipher-настроек зависит от версии OpenVPN и политики безопасности организации

В базовой лаборатории важнее правильно собрать туннель, маршруты и проверку доступа

Нельзя механически копировать production-конфигурацию без понимания версии сервера и клиента

Криптографическое усиление выносится в продвинутый блок

Client profile описывает подключение client01

client

dev tun

proto udp

remote 172.16.100.10 1194

nobind

persist-key и persist-tun сохраняют ключ и туннель при
переподключении

Client profile содержит материалы доступа

remote-cert-tls server

ca ca.crt

cert client01.crt

key client01.key

tls-crypt ta.key

verb 3

.ovpn-профиль нужно хранить как секрет

client profile содержит данные доступа к VPN

Если в .ovpn встроить сертификаты и ключ, сам файл становится полноценным credential

Inline profile удобен для импорта клиентом, но повышает требования к хранению файла

Файл нельзя отправлять в общий чат или хранить в открытой папке

При утечке client01.ovpn сертификат client01 нужно отозвать

Push Route добавляет маршрут к внутренней сети

Директива сервера: `push "route 192.168.30.0
255.255.255.0"`

После подключения клиент проверяет маршрут командой `ip route`

Ожидаемо: `192.168.30.0/24` идет через VPN/tun

Если маршрута нет, клиент не знает, куда отправлять трафик к `web01`

Route не разрешает доступ сам по себе: его еще ограничивает firewall

DNS Push помогает обращаться к внутренним именам

Директива: `push "dhcpcd-option DNS 192.168.30.53"`

Реальное применение DNS зависит от клиентской ОС и VPN-клиента

Проверка имени: `getent hosts web01.company.test`

Если по IP `web01` доступен, а по имени нет, проверяют DNS на клиенте

`NetworkManager`, `systemd-resolved` и `update-resolv-conf` в этой лекции не разбираются подробно

Split Tunnel направляет через VPN только нужную сеть

Split Tunnel - через VPN идет только 192.168.30.0/24

Остальной трафик клиента остается через обычный локальный gateway

В этой лаборатории используется Split Tunnel

Full Tunnel направляет весь трафик клиента через VPN

Full Tunnel задается директивой push "redirect-gateway def1", но здесь не настраивается

Схема: Split Tunnel к сети 192.168.30.0/24

5. IP forwarding разрешает vpn01 пересылать пакеты

`sysctl net.ipv4.ip_forward` - проверяет, включена ли IPv4-пересылка

`sudo sysctl -w net.ipv4.ip_forward=1` - включает forwarding временно

`echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-openvpn-forward.conf` - сохраняет настройку

`sudo sysctl --system` - применяет sysctl-файлы

Без `ip_forward` `vpn01` примет VPN-пакет, но не перешлет его к `web01`

В лаборатории используем NAT для возвратного трафика

10.8.0.0/24 → vpn01 → masquerade → 192.168.30.0/24

Плюс NAT: на web01 или внутреннем gateway не нужен отдельный route к 10.8.0.0/24

Минус NAT: web01 видит адрес vpn01, а не реальный VPN IP клиента

Для учебной лаборатории NAT проще и надежнее

Маршрутизация без NAT лучше подходит для production-аудита и точного учета клиентов

Минимальный пример NAT на nftables

```
sudo nft add table ip nat
```

```
sudo nft 'add chain ip nat postrouting { type nat hook  
postrouting priority srcnat; }'
```

```
sudo nft add rule ip nat postrouting oifname "enp0s8" ip  
saddr 10.8.0.0/24 ip daddr 192.168.30.0/24 masquerade
```

Правило NAT ограничено VPN-подсетью и внутренней сетью

Синтаксис nftables здесь нужен как практический минимум, а не отдельный курс firewall

Firewall нужно разделять на INPUT и FORWARD

INPUT - трафик к самому vpn01

FORWARD - трафик через vpn01 между интерфейсами

UDP 1194 allowed in INPUT не означает доступ к web01

Доступ к web01 контролируется FORWARD-правилами

Перед изменением firewall нужно понять,
используется ли UFW, firewalld или nftables

Правило доступа должно разрешать HTTPS и запрещать SSH

VPN client → web01 HTTPS → PERMIT

VPN client → web01 SSH → DENY

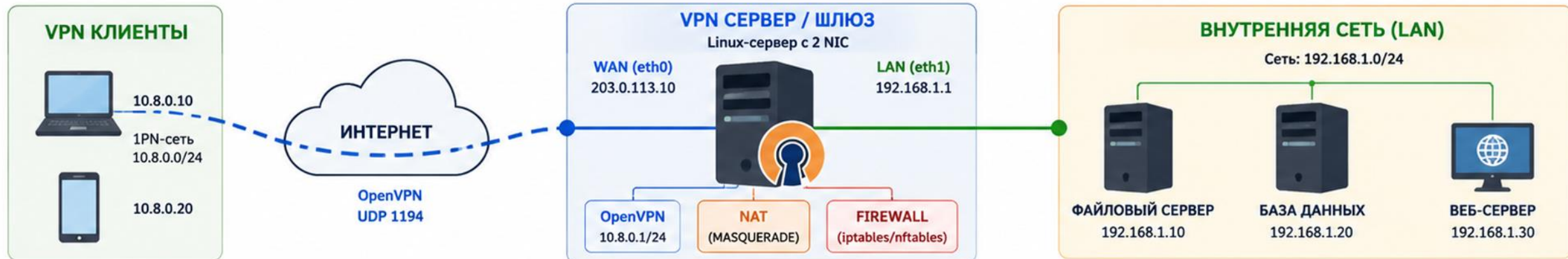
Пример permit: `sudo nft add rule inet filter forward
iifname "tun0" oifname "enp0s8" ip saddr 10.8.0.0/24 ip
daddr 192.168.30.10 tcp dport 443 accept`

Возвратный трафик должен разрешаться как
`established,related`

SSH не добавляется в разрешающие правила, чтобы
доступ к web01:22 оставался закрытым

СХЕМА: МАРШРУТИЗАЦИЯ, NAT И FIREWALL ДЛЯ OPENVPN

Безопасный доступ клиентов VPN к внутренней сети



ПУТЬ ПАКЕТА



ПРАВИЛА FIREWALL (MASQUERADE GW)

- ✓ Разрешить: UDP 1194 (OpenVPN)
- ✓ Разрешить: трафик из VPN-сети 10.8.0.0/24 в LAN 192.168.1.0/24
- ✓ Разрешить: ответный трафик (ESTABLISHED, RELATED)
- ✗ Запретить: доступ из VPN в Интернет (если не требуется)
- ✗ Запретить: доступ из LAN в VPN-сеть

1. МАРШРУТИЗАЦИЯ

- Клиенты VPN получают адреса из сети 10.8.0.0/24
- На VPN-сервере включена IP-перееадресация
- Маршрут к сети 10.8.0.0/24 идет через OpenVPN
- Маршрут к LAN (192.168.1.0/24) через eth1

```
sysctl -w net.ipv4.ip_forward=1
# Проверка
sysctl net.ipv4.ip_forward
# Ожидаемый вывод: net.ipv4.ip_forward = 1
```

2. NAT (MASQUERADE)

- Для доступа клиентов VPN в Интернет
- Исходящий трафик подменяет источник на внешний IP сервера (203.0.113.10)
- Для доступа в LAN NAT обычно не нужен

```
iptables -t nat -A POSTROUTING -s 10.8.0.0/24 \
-o eth0 -j MASQUERADE
# Проверка
iptables -t nat -L -n -v
```

3. FIREWALL (ФИЛЬТРАЦИЯ)

- Разрешаем только необходимый трафик
- Закрываем доступ к VPN-сети из LAN
- Применяем правило по умолчанию DROP

```
# Пример политики по умолчанию
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT
```

4. ИТОГ

- Клиенты VPN безопасно получают доступ к ресурсам внутренней сети
- NAT позволяет выход в Интернет (при необходимости)
- Firewall защищает сервер и внутреннюю сеть от нежелательного доступа
- Все соединения управляются политиками маршрутизации и безопасности

УСЛОВНЫЕ ОБОЗНАЧЕНИЯ

- VPN-туннель (шифрованный)
- Внутренняя сеть (LAN)
- Внешняя сеть / Интернет
- Маршрутизация
- Обратный путь (ответ)

ПРИМЕР КОНФИГУРАЦИИ OPENVPN (server.conf)

```
port 1194
proto udp
push "route 192.168.1.0 255.255.255.0"
dev tun
server 10.8.0.0 255.255.255.0
persist-key
persist-tun
```



СЕТЕВАЯ СВОДКА

VPN-сеть: 10.8.0.0/24
LAN-сеть: 192.168.1.0/24
WAN (публичный IP): 203.0.113.10
OpenVPN порт: UDP 1194

СОВЕТ ПО БЕЗОПАСНОСТИ

- Используйте TLS-шифрование, сертификаты, сложные ключи и регулярно обновляйте систему.

Диагностика OpenVPN-подключения по уровням

Проверяйте сверху вниз — остановитесь на уровне, где найдено несоответствие.

УРОВЕНЬ		ЧТО ПРОВЕРЯЕМ	КАК ПРОВЕРИТЬ (примеры)	ОЖИДАЕМЫЙ РЕЗУЛЬТАТ	ЕСЛИ НЕ РАБОТАЕТ
L1		Сеть и доступность VPN-сервера - IP-адрес и маршрут - Порт UDP/TCP 1194	<pre>ping <vpn_server_ip> traceroute <vpn_server_ip> nc -vz -u <vpn_server_ip> 1194 # или: telnet <vpn_server_ip> 1194</pre>	 Узел доступен. Маршрут проходит. Порт 1194 открыт.	 Проблема с сетью/ маршрутизацией. Порт закрыт фаерволом или провайдером.
L2		Сервис OpenVPN на сервере - Процесс запущен - Слушает нужный порт	<pre>systemctl status openvpn-server@server ss -ltnp grep 1194 journalctl -u openvpn-server@server -n 50</pre>	 Сервис active (running). Слушает UDP/TCP 1194.	 Сервис не запущен или слушает другой адрес/порт.
L3		TLS-рукопожатие и шифрование - Сертификаты CA/Server - Версия TLS, шифры	<pre>openssl s_client -connect <vpn_server_ip>:1194 -tls1_2 # или: -tls1_3 / -udp (если поддерживается) # на клиенте: openvpn --verb 5 --connect-retry 2</pre>	 TLS handshake успешен. Нет ошибок сертификатов. Договорились алгоритмы.	 Ошибки сертификатов, TLS handshake failed, неверные параметры шифрования.
L4		Аутентификация клиента (сертификат/логин) - Проверка сертификата - Права клиента (CN)	<pre># на сервере: grep -E "AUTH VERIFY CN=" /var/log/openvpn.log # на клиенте в конце лога: Initialization Sequence Completed</pre>	 Client 'CN' authenticated или Initialization Sequence Completed	 AUTH_FAILED, VERIFY ERROR, клиент не допущен.
L5		VPN-интерфейс и IP - Интерфейс tun0/tap0 - IP-адрес из пула VPN - Маршруты на клиенте	<pre># клиент: ip addr show tun0 ip route show # сервер: ip addr show tun0</pre>	 tun0 up. IP из пула (напр. 10.8.0.x). Маршруты установлены.	 Нет IP на tun0. Маршрут к внутренней сети не установлен.
L6		Маршрутизация, NAT и firewall на сервере - IP forwarding - Правила firewall - NAT (при доступе в Internet)	<pre>sysctl net.ipv4.ip_forward iptables -L -n -v iptables -t nat -L -n -v</pre>	 net.ipv4.ip_forward = 1 Правила разрешают трафик. NAT работает (если нужен выход в Internet).	 Трафик запрещён firewall. Нет маршрута к сети. IP forwarding выключен.
L7		Доступ к внутреннему ресурсу (web01) - Маршрут до 192.168.30.10 - Сервис на хосте	<pre>ping 192.168.30.10 curl http://192.168.30.10 nc -vz 192.168.30.10 80</pre>	 Пинг/HTTP успешны. Сервис отвечает.	 Хост недоступен, сервис не запущен, блокирует firewall хоста.

БЫСТРЫЕ ДЕЙСТВИЯ



Нет соединения с сервером

→ Проверьте сеть, маршрут и порт 1194 (UDP/TCP)



Сервис не слушает

→ Запустите сервис OpenVPN и проверьте порт



Ошибка TLS/сертификатов

→ Проверьте CA, сертификаты, даты и имена (CN/SAN)



AUTH_FAILED

→ Проверьте клиентский сертификат и права (CN)



Нет IP на tun0

→ Проверьте пул адресов и маршруты на клиенте



Не проходит трафик

→ Разрешите forwarding и правила firewall, при необходимости NAT



Ресурс недоступен

→ Проверьте маршрут к сети и запущенный сервис



Совет: включите подробные логи на клиенте: `openvpn --verb 5` и на сервере: `verb 4-5`

УСПЕШНОЕ ПОДКЛЮЧЕНИЕ:



VPN-туннель
(10.8.0.0/24)



vpn01
(OpenVPN)



Internal LAN
(192.168.30.0/24)



web01
(192.168.30.10)

Обозначения:

-  Проверка пройдена
-  Требуется внимания
-  Проблема найдена

После подключения сначала проверяют tun и route

`ip -br addr` - показывает, появился ли VPN-интерфейс

Ожидаемо: у клиента есть адрес из 10.8.0.0/24

`ip route` - показывает маршрут к 192.168.30.0/24

Если tun есть, но маршрута нет, проверяют `push route`

Если маршруты есть, переходят к проверке web01

VPN считается рабочим только после проверки сервиса

Разрешенный сценарий: `curl -vk https://192.168.30.10/`

Ожидаемый результат: клиент получает HTTPS-ответ от web01

Запрещенный сценарий: `nc -vz 192.168.30.10 22`

Ожидаемый результат: SSH-доступ к web01 не проходит

Connected в клиенте OpenVPN не доказывает, что нужная политика доступа работает

6. CRL блокирует ранее выданный сертификат

Certificate Revocation List - список отозванных сертификатов

Client Certificate → revoke → CRL → new connection denied

server.conf: `crl-verify /etc/openvpn/server/crl.pem`

CRL нужна, если client01 потерян, уволен или профиль утек

В этой лекции CRL рассматривается базово, без глубокого разбора активных сессий

Диагностика OpenVPN идет по уровням

1. OpenVPN service → 2. UDP 1194 listening
3. UDP packets reach server → 4. TLS/certificate
5. tun interface → 6. client route
7. IP forwarding → 8. NAT/firewall
9. web01 HTTPS

Если пропустить уровень, можно лечить firewall при ошибке сертификата или наоборот

Команды диагностики OpenVPN должны идти по порядку

`systemctl status openvpn-server@server` - состояние службы

`journalctl -u openvpn-server@server -b` - журнал OpenVPN

текущей загрузки

`ss -ltnr | grep 1194` - слушается ли UDP 1194

`tcpdump -nn -i enp0s3 udp port 1194` - доходят ли UDP-
пакеты до `vpn01`

`ip -br addr show tun0` и `ip route` - есть ли туннель и маршрут

`sysctl net.ipv4.ip_forward` и `curl -vk https://192.168.30.10/` -
`forwarding` и доступ к сервису

Конфликт сетей ломает Remote Access routing

Home LAN: 192.168.30.0/24

Corporate LAN: 192.168.30.0/24

Если домашняя и корпоративная сети совпадают, клиент может выбрать локальный маршрут вместо VPN

Симптом: VPN connected, но web01 по 192.168.30.10 не открывается

Главный вывод: пересечение адресных пространств ломает Remote Access routing

Дополнительно: CCD задает индивидуальную политику клиентов

CCD (Client-Specific Configuration) - индивидуальная конфигурация для конкретного клиентского сертификата

OpenVPN может выдавать отдельные параметры разным клиентам

CCD полезен для индивидуальных маршрутов и ограничений

В этой лекции команды CCD не разбираются

Главная лабораторная цель - один client01 и один доступ к web01 HTTPS

Единый практический сценарий: настройка

1. Install OpenVPN
2. Create basic PKI
3. Issue vpn01 certificate
4. Issue client01 certificate
5. Configure server.conf
6. Enable forwarding
7. Configure NAT/firewall

Единый практический сценарий: проверка

8. Create client01.ovpn
9. Connect
10. Verify tun/route
11. Test HTTPS
12. Test SSH deny
13. Revoke client01
14. Confirm new connection denied

ДИАГРАММА: ДИАГНОСТИКА OPENVPN-ПОДКЛЮЧЕНИЯ ПО УРОВНЯМ



🎯 Принцип: проверяем **снизу вверх** — от сети до приложений

УРОВЕНЬ	ЧТО ПРОВЕРЯЕМ	КАК ПРОВЕРИТЬ (ПРИМЕРЫ)	ОЖИДАЕМЫЙ РЕЗУЛЬТАТ	ЕСЛИ НЕ РАБОТАЕТ	ПОРЯДОК ДИАГНОСТИКИ
6 ПРИЛОЖЕНИЯ (СЕРВИСЫ) 	Доступ к внутренним ресурсам и сервисам через VPN	<code>ping 10.8.0.10</code> <code>telnet 10.8.0.10 3389</code> <code>curl http://10.8.0.10</code> <code>traceroute 10.8.0.10</code>	✅ Ресурсы доступны, сервисы отвечают, маршрут до хоста проходит через VPN-интерфейс	⚠️ Проверить сервисы на сервере Проверить межсетевой экран (FW) Проверить маршрутизацию на сервере Проверить правила доступа	6 Доступны ли внутренние ресурсы? ↑
5 VPN-ТУННЕЛЬ (OPENVPN) 	Состояние OpenVPN-клиента и туннеля, получение IP-адреса, обмен пакетами	<code>openvpn --status</code> <code>ip addr show tun0</code> <code>route grep 10.8.0.0</code> <code>logread grep -i openvpn</code>	✅ Клиент подключён (CONNECTED) IP-адрес назначен Трафик идёт через tun-интерфейс Нет ошибок в логах	⚠️ Проверить конфиг клиента Проверить сертификаты и ключи Проверить логи OpenVPN Проверить время на клиентах	5 Поднялся ли VPN-туннель? ↑
4 ХОСТ / ОС (ЛОКАЛЬНЫЙ УРОВЕНЬ) 	Интерфейсы, маршруты, DNS, фаервол хоста	<code>ip addr</code> <code>ip route</code> <code>resolvectl status / cat /etc/resolv.conf</code> <code>iptables -L / nft list ruleset</code>	✅ Интерфейс tun0 UP Маршруты на сети VPN есть DNS работает Фаервол не блокирует трафик	⚠️ Добавить/исправить маршруты Исправить DNS Открыть нужные порты/правила FW	4 Всё ли в порядке на хосте? ↑
3 ТРАНСПОРТ (TCP / UDP) 	Доступность порта OpenVPN (по умолчанию UDP 1194) до VPN-сервера	<code>nc -zv <vpn-server> 1194 -u</code> <code>telnet <vpn-server> 1194</code> <code>nmap -sU -p 1194 <vpn-server></code>	✅ Порт 1194/UDP открыт и доступен Ответ от сервера есть (или порт отображается как open)	⚠️ Проверить фаервол/фаервол провайдера Проверить порт и протокол в конфиге Проверить NAT/Port Forwarding	3 Доступен ли порт OpenVPN? ↑
2 СЕТЬ (IP) (МАРШРУТИЗАЦИЯ) 	Маршрутизация до VPN-сервера, доступность по IP-адресу	<code>ping <vpn-server-ip></code> <code>traceroute <vpn-server-ip></code> <code>ip route get <vpn-server-ip></code>	✅ Есть маршрут до сервера Пакеты доходят (ответ есть) Нет потерь на пути	⚠️ Проверить маршрут по умолчанию Проверить промежуточные роутеры Проверить доступ в интернет	2 Есть ли маршрут до VPN-сервера? ↑
1 ФИЗИЧЕСКИЙ УРОВЕНЬ (L1) 	Физическое подключение и базовая связь с сетью	<code>ip link show</code> <code>ethtool eth0</code> <code>ping 8.8.8.8</code>	✅ Интерфейс UP, линк активен Есть связь с внешней сетью Нет ошибок линка	⚠️ Проверить кабель/Wi-Fi Проверить драйвер/адаптер Обратиться к провайдеру	1 Есть ли физическая связь? ↑
 Исправляйте проблему на текущем уровне, затем поднимайтесь выше					

 **БЫСТРЫЕ ПРОВЕРКИ**

 `openvpn --status`
Быстрый статус клиента

 `ip addr show tun0`
Проверка IP и интерфейса

 `route | grep 10.8.0.0`
Проверка маршрута через VPN

 `logread | grep -i openvpn`
Ошибки и причина разрыва

 **ЧАСТЫЕ ПРИЧИНЫ ПРОБЛЕМ**

- Неверный конфиг (IP, порт, протокол)
- Истёкшие сертификаты / неверное время
- Блокировка портов фаерволом / провайдером
- Отсутствие маршрута до VPN-сети

 **ПОЛЕЗНО ЗНАТЬ**

Логи OpenVPN – главный источник информации. Всегда начинайте с них.

Итоги лекции

Remote Access VPN подключает отдельного клиента к внутренней сети

В лаборатории OpenVPN работает через UDP 1194, TUN и подсеть 10.8.0.0/24

PKI нужна для проверки сервера и каждого отдельного клиента

push route и DNS push помогают клиенту найти внутреннюю сеть и имена

ip_forward, NAT и FORWARD-правила позволяют контролировать доступ к web01

Успех доказывается не статусом Connected, а доступом к HTTPS и запретом SSH

Контрольные вопросы

1. Для чего нужен OpenVPN?
2. Чем Remote Access отличается от Site-to-Site?
3. Почему используется TUN?
4. Для чего нужен CA?
5. Зачем каждому клиенту отдельный сертификат?
6. Что делает push "route ..."?
7. Для чего нужен ip_forward?
8. Чем INPUT отличается от FORWARD?
9. Что такое Split Tunnel?
10. Как проверить, что клиент реально получил доступ к web01?

СХЕМА: ПУТЬ ПАКЕТА ЧЕРЕЗ OPENVPN К ВНУТРЕННЕМУ СЕРВЕРУ



- 1** Клиент формирует пакет для 192.168.10.20
Источник: 10.8.0.10 (VPN IP клиента)
- 2** Пакет шифруется и отправляется в OpenVPN-сервер
Через интернет по UDP 1194
- 3** Сервер расшифровывает пакет
Проверяет маршрут до 192.168.10.20
- 4** Пакет отправляется во внутреннюю сеть
Через LAN-интерфейс (192.168.10.1)

- 5** Внутренний сервер получает пакет
IP назначения: 192.168.10.20
- 6** Ответ от сервера к клиенту
Идёт обратно к VPN-серверу
- 7** Сервер шифрует ответ и отправляет в интернет
Клиенту на его внешний адрес
- 8** Клиент расшифровывает ответ и принимает его
Соединение установлено, обмен данными

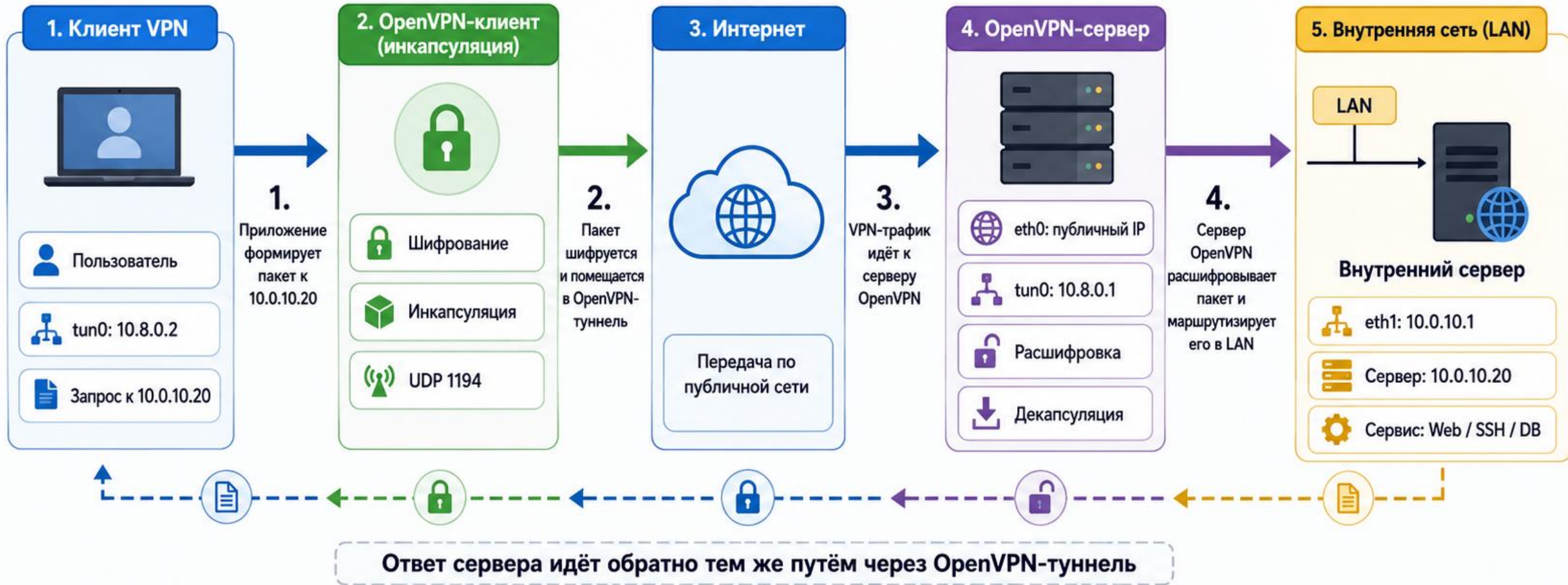
ПРИМЕР АДРЕСАЦИИ

- VPN-клиент: 10.8.0.10
- VPN-сервер (WAN): 203.0.113.5
- VPN-сервер (LAN): 192.168.10.1
- Внутренняя сеть: 192.168.10.0/24
- Внутренний сервер: 192.168.10.20

ВАЖНО

- Трафик между клиентом и сервером защищён шифрованием.
- Маршруты до внутренних сетей указываются на клиенте (push или настройка маршрутов).
- Межсетевой экран на сервере должен разрешать трафик из VPN к внутренней сети.

Схема: путь пакета через OpenVPN к внутреннему серверу



Обозначения

- Исходный (открытый) пакет данные приложения
- Зашифрованный VPN-трафик (внутри OpenVPN-туннеля)
- Дешифрованный пакет (исходные данные)

Что важно для работы

- Маршрут к сети 10.0.10.0/24 через tun0
- Доступ в firewall разрешён
- OpenVPN-процесс слушает UDP 1194
- У внутреннего сервера есть обратный маршрут к 10.8.0.0/24